DevSecure: Autonomous, Fail-Closed SAST Remediation

AI proposes. AI critiques. Determinism decides.

COLOR SEMANTICS

- Deterministic systems
- Orchestration & policy
- LLM agents
- Verified success
- Diversity / independent LLM
- Fail-closed discard paths

The Problem with Today's SAST

Findings

- Alerts
- Detections
- Vulnerabilities

Current State

- Manual Fixes
- Unsafe Auto-Fix
- Human Bottlenecks

- Alerts without safe remediation
- Human review does not scale
- Auto-fix without validation
- Auto-fixes that pass CI but silently alter security semantics (e.g., auth, validation logic)

DevSecure High-Level Architecture (Hero Diagram)

Remediation Control Plane (Autonomy & Policy Routing)

3-Tier CWE Resolution

- **T1** Coarse mapping (category level)
- **T2** Specific CWE identification
- **T3** Precise context & variant

26-Dimensional Complexity Scoring 26D

Only a small core of routing dimensions is computed synchronously; remaining signals are evaluated asynchronously for complex lanes.

- Code context depth
- Dataflow complexity
- Test coverage
- Blast radius
- Language idioms
- Coupling degree
- Semantic density
- Change frequency
- API surface area
- Security criticality
- Type complexity
- Control flow depth
- +14 more dimensions

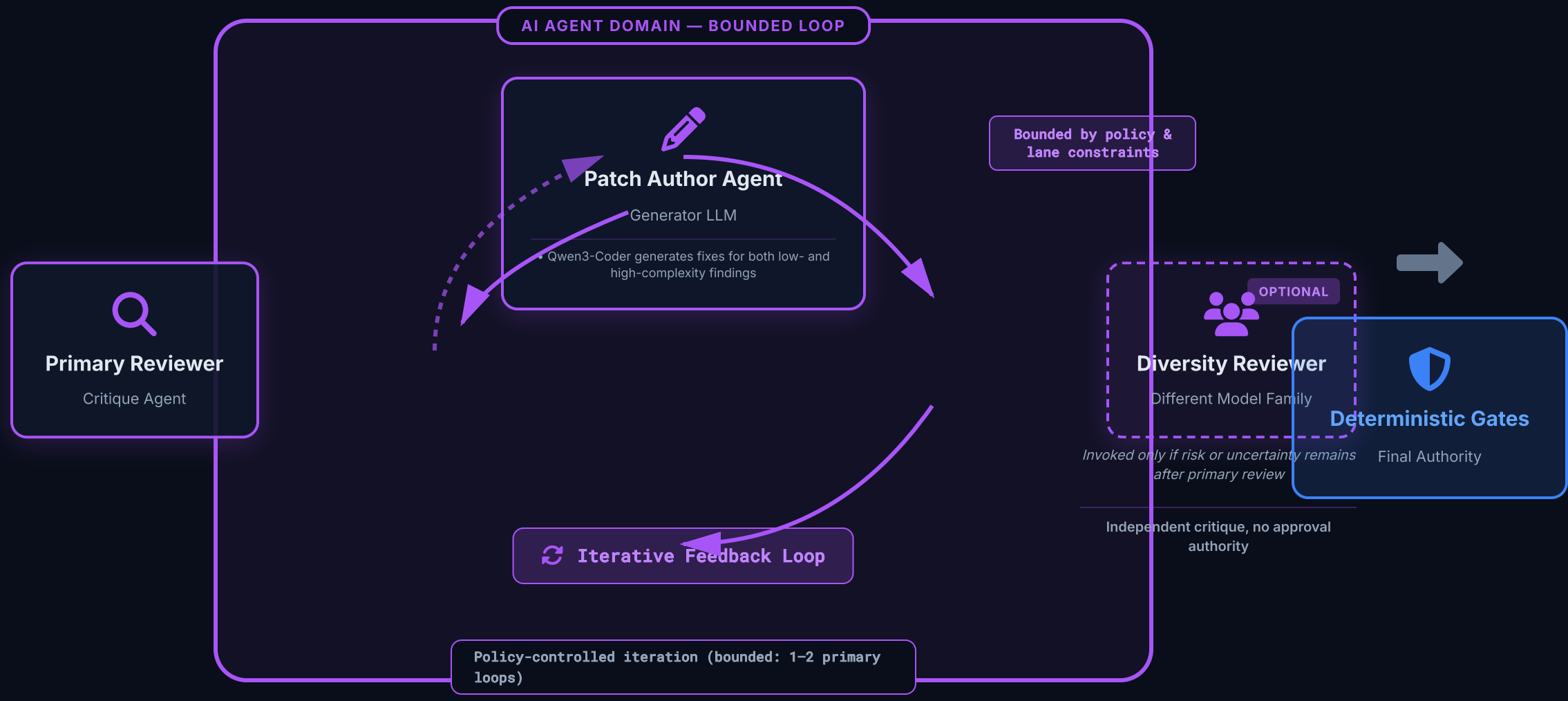
Lane Routing (L1–L4)

- **L1** Trivial patterns, minimal context
- **L2** Localized changes, single-file
- **L3** Multi-file, dataflow analysis
- **L4** High-risk, semantics-heavy

Not every vulnerability is equal — routing matters.

If routing telemetry is unavailable or times out, findings default to L4 manual handling.

Autonomous Remediation Loop (Agentic Core)



Qwen3-Coder may detect and propose fixes, but findings become actionable only after deterministic corroboration.

Diversity Review (Explicit Callout)

ALWAYS-ON

Reviewer #1

Primary Baseline Reviewer

- Always-on, cost-efficient operation
- Fast baseline critique
- Consistent model family
- Optimized for throughput

+

CONDITIONAL

Reviewer #2

Diversity Reviewer

- Triggered by complexity, disagreement, or unresolved risk after primary review
- Different model family/vendor
- Breaks correlated blind spots
- Independent architecture lineage
- Invoked on <10–15% of findings (risk-gated)

DIFFERENT MODEL FAMILY

Break correlated failure modes without human escalation.

Diversity review reduces correlated LLM failure modes without increasing human or cost overhead

Diversity review reduces correlated model error, not reviewer authority

Deterministic Verification Gates (Final Authority)

Explicit Non-Goals

- No self-approving AI
- No unbounded retries
- No human override path
- No learning from unverified fixes

These constraints are architectural, not configurable.

DETERMINISTIC DOMAIN

Required Verification Gates

NON-NEGOTIABLE CHECKS

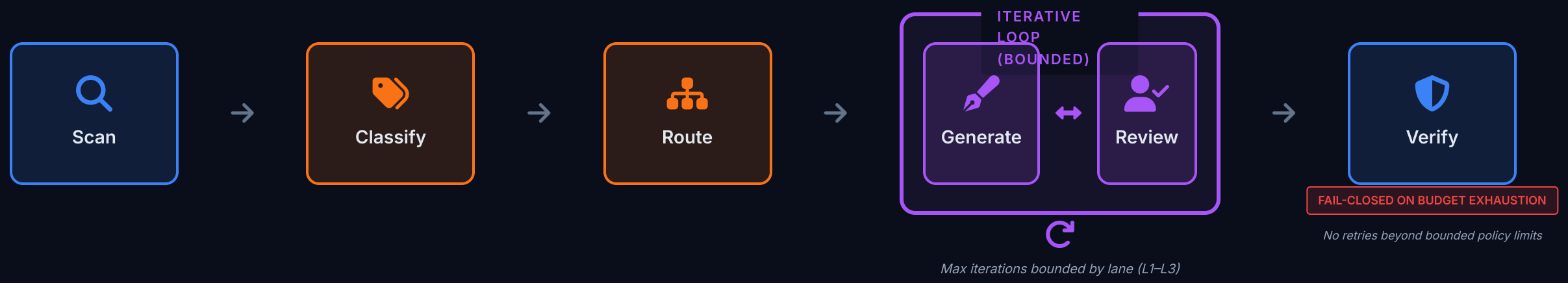
- Re-run the original detector that produced the finding, or the LLM detection harness plus its deterministic corroborator. * REQUIRED
  - *For LLM-detected findings, removal is proven by re-running the same corroboration harness after the fix.*
- Build & tests pass * REQUIRED
- Semantics preserved * REQUIRED
- Minimal diff * REQUIRED
  AST-normalized diff removes formatting-only and non-functional changes before verification.

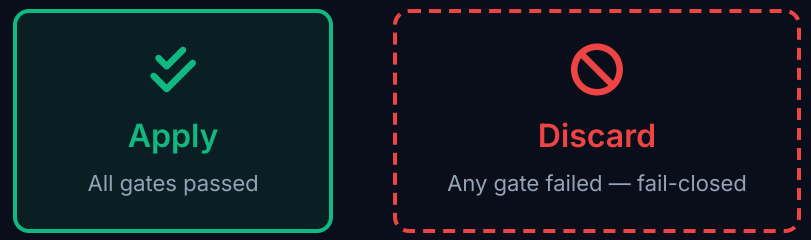
Fast, targeted verification gates run per iteration; full test suites are required only for final PR application.

Targeted tests are selected via dependency graph analysis to execute only those impacted by the AST diff.

Gate failure triggers feedback into the bounded remediation loop.
Fail-closed discard occurs only after the iteration budget is exhausted.

End-to-End Control Flow





Continuous Learning (Verified Only)

ORANGE CONTROL VALIDATES PROVENANCE

Input Sources

External & internal advisories

- GHSA (GitHub Security Advisories)
- Internal security reports
- CVE/CWE databases
- FreshFix (GHSA mining): weekly verified before/after patches
- Used to enrich Qwen3-Coder RAG prompts, not direct weight updates
- Deterministic gate failures emit structured telemetry that is fed back into the next generation prompt, without bypassing any gate.

DETERMINISTIC TELEMETRY ONLY

Compiler errors and test failures are fed back into the generator's RAG context. No opinions, confidence scores, or failed patches enter learning.

Verified Commits

Gate-approved fixes only

100% VERIFIED

- Vuln removed
- Tests pass
- Semantics OK
- Gates passed

Gold Dataset

Curated training & evaluation corpus

- Training prompts & examples
- Evaluation benchmarks
- Continuous corpus update

NO RED PATHS FEED LEARNING

The system learns only from proven fixes.

Competitive Differentiation

	Others	Typical Auto-Fix	DevSecure (OUR APPROACH)
AI ARCHITECTURE	Single LLM	Binary judge	Agentic debate
REVIEW PROCESS	No validation	Best-effort	Deterministic gates
APPROVAL MECHANISM	Manual review	Risky automation	Fail-closed
SAFETY MODEL	Human-dependent	Unsafe by default	Safe by design
SCALABILITY	Limited by humans	Fast but risky	Autonomous & safe

Why This Is Defensible

STRATEGIC MOAT

Architectural Defensibility

- Multi-agent architecture — MOAT
- Deterministic authority — MOAT
- Diversity review — MOAT
- Verified learning loop — MOAT
- Architectural enforcement survives model churn and vendor changes — MOAT

The moat comes from composability + deterministic enforcement.

Autonomous Security Remediation — Done Safely

SAFE BY DESIGN

This is how **security scales** without breaking **trust**.

COLOR LEGEND: Deterministic · LLM Agents · Orchestration · Verified · Fail-Closed

Architect Q&A — Hard Questions Answered

BACKUP SLIDE

Technical deep-dive for architectural review

How many LLM iterations occur before diversity review?

Two primary review iterations by default. Diversity review is invoked only if risk, uncertainty, or sensitive code paths remain.

Can an LLM ever approve a fix?

No. LLMs provide critique only. Deterministic verification gates are the sole approval authority.

What prevents infinite loops?

Lane-based iteration budgets (L1–L3). If convergence is not achieved, the patch is discarded.

How do you control cost at scale?

Diversity review is risk-gated and invoked on <10–15% of cases. Most fixes complete with a single reviewer.

What happens if models degrade or vendors change?

Models are replaceable. Deterministic gates, routing policy, and enforcement logic remain unchanged.

DevSecure Routing Layers — Execution vs Control

Four distinct architectural layers with clear separation of concerns

L1 Execution Plane: DevSecure Security Router v2.0

SCANNER ORCHESTRATION

Go Router | Multi-heuristic file identification | Conditional Lizard (SAST only)

CCN ≤ 15 → OpenGrep / Semgrep

Fast, deterministic pattern-based SAST detection

CCN > 15 → Qwen3-Coder

Deep LLM-based static analysis for complex code

LLM-detected findings require deterministic corroboration before remediation eligibility.

RabbitMQ dispatch | PostgreSQL storage

Routes files to the correct security scanners. No remediation logic.

L2 Classification Plane: CWE Classification Pipeline

VULNERABILITY UNDERSTANDING

SAST Finding + Code Snippet → Qwen2.5-Coder-1.5B (Advisory SLM) → JSON Schema Guardrail (Strict Enforcement)

→ OUTPUT: | CWE ID | Relative Confidence | Complexity Flags (26D Core)

Single-pass advisory SLM extracts classification and routing signals. Uncertainty strictly defaults to conservative manual routing (Fail-Closed).

L3 Control Plane: Remediation Control Router

AUTONOMY GOVERNANCE

End-to-End Routing & Governance Architecture

From file ingestion to enforced remediation — with clear separation of concerns

File Ingestion
Source code input

→

Execution Plane
Scanner Orchestration

→

Classification Plane
CWE Understanding

→

Control Plane
Remediation Governance

→

Agentic Remediation
AI proposes & critiques

→

Deterministic Gates
Final authority

→

Outcome
Applied / Discarded

KEY TAKEAWAY

DevSecure separates **execution routing**, **vulnerability understanding**, and **remediation governance** into distinct architectural layers — ensuring performance, accuracy, and safety without coupling or ambiguity.

DevSecure: Autonomous, Fail-Closed SAST Remediation

AI proposes. AI critiques. Determinism decides.

Autonomous security remediation without human bottlenecks or unsafe automation

The Problem: Security Does Not Scale

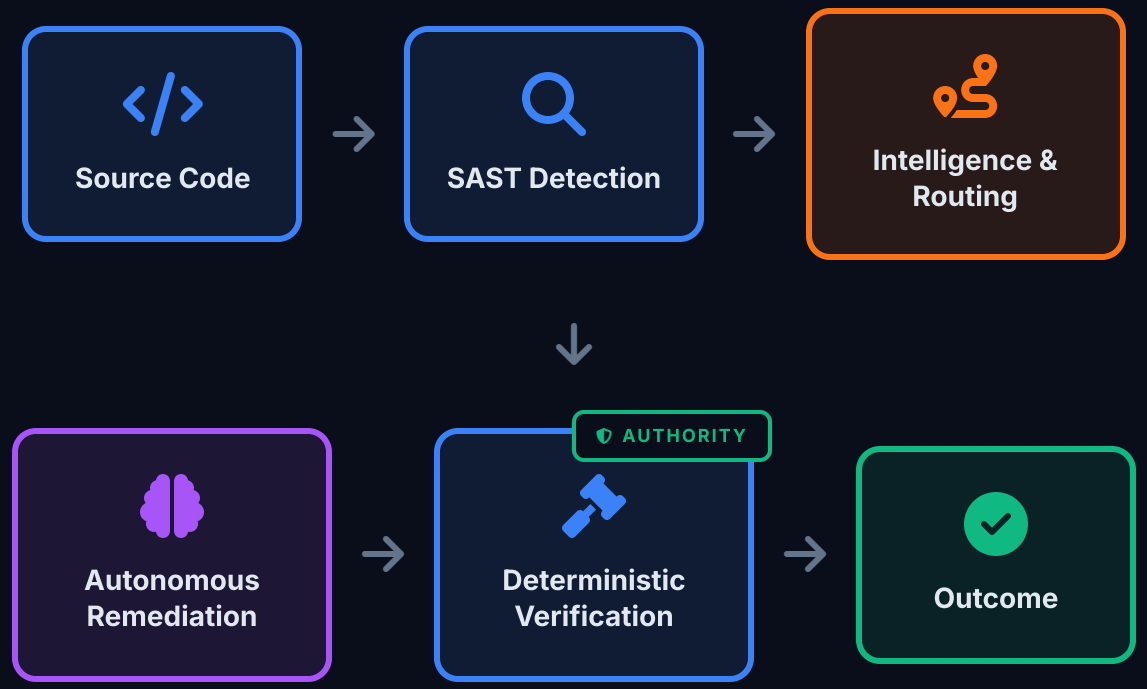
Today's SAST Reality

- Findings without safe remediation
- Manual fixes do not scale
- Auto-fixes alter security semantics
- Human review becomes the bottleneck

! CORE PROBLEM

Security teams must choose between speed and safety — and lose either way.

DevSecure at a Glance



End-to-end autonomous remediation with deterministic approval authority

AI assists — deterministic systems decide.

Autonomous Remediation, Done Safely

Agentic Remediation Loop

- Patch Author AI generates minimal fixes
- Primary Reviewer AI critiques like a senior engineer
- Independent Diversity Reviewer invoked only on risk
- Iterations are bounded and policy-controlled

KEY BENEFIT

Two independent AI reviewers. Zero human escalation.

Fail-Closed by Design

Deterministic Verification Gates

- Vulnerability fully removed — REQUIRED
- Builds and tests pass — REQUIRED
- Semantics preserved — REQUIRED
- Minimal diff enforced — REQUIRED

FAIL-CLOSED

If any gate fails, the fix is discarded.

Why DevSecure Is Different

Others

- Alerts only
- Human dependent
- Slow remediation

Typical Auto-Fix

- Single AI suggestion
- Binary approval
- Unsafe by default

DevSecure

- Agentic debate
- Deterministic enforcement
- Safe autonomy at scale

This is not auto-fix — it is autonomous remediation.

Defensible by Architecture

- Multi-agent AI architecture
- Deterministic approval authority
- Diversity review to prevent correlated failures
- Verified learning loop only from proven fixes

FINAL MESSAGE

Autonomous security remediation, enforced by architecture.

READY TO SCALE